LEARNING MANAGEMENT SYSTEM

Student: T. MAFATLANE; Student No.: 202321187; Submission date: 2 November 2026

1. Project overview and engineering objectives

This project developed a desktop Moderated Learning Management System (LMS) for managing courses, assignments and student assessment. It supports four roles Student, Lecturer, Head of Department (HoD) and Administrator with role-specific access. The central control is that lecturer-entered marks remain unavailable to students until they have been moderated and released by an authorized HoD. The system also supports course administration, enrolment, assignment creation, submission and resubmission, individual extensions, marking, written feedback and audit records.

The engineering objective was to deliver a persistent, secure and maintainable C++ application while following the CS4433 software life cycle. The implemented environment uses C++17, Qt 6 Widgets, SQLite, CMake/Ninja, GCC, Argon2id password hashing, GoogleTest and CTest. The application is designed for offline/local operation; performance at the specified 800-student scale and deployment on the school's oldest hardware still require dedicated verification.

2. Development process and project planning

The project plan adopted a seven-week Waterfall structure, with phase gates and V-Model verification. Work was divided into planning and feasibility, requirements engineering, system design, implementation, verification and validation, deployment, and maintenance. Dependencies were managed through the critical path: Planning → Requirements → Gate 1 → Design → Gate 2 → Implementation → Verification and Validation → Gate 3 → Deployment → Maintenance → Completion. The single-developer schedule and the dependency on design approval made early requirements clarification and incremental testing important.

The feasibility study found the project technically, operationally, economically, legally and schedule feasible, subject to resource confirmation, performance testing, security controls and user training. Principal risks included legacy hardware, the 800-student dataset, credential and assessment-data exposure, requirements conflict, schedule compression and teacher usability. Risk exposure was assessed as likelihood × impact, with reviews planned at each phase gate.

Figure 1. Work Breakdown Structure and project schedule. View diagram

Figure 2. Project schedule. View diagram

3. Requirements engineering

The SRS defines the Moderated Learning Management System's scope, actors, functional requirements, measurable quality requirements and business rules. Stakeholders include students, lecturers, HoDs, the principal, system administrator, IT officer and developer. The principal's requirement for controlled publication conflicted with the lecturer's preference for immediate visibility. The resolution is a two-stage process: lecturers mark and provide feedback; HoDs review, approve or return marks; only an explicit release makes results visible to students.

Other conflicts were resolved by accepting late work subject to a defined penalty rather than rejecting it automatically; permitting resubmission before the applicable deadline while retaining submission history; and allowing individual lecturer-approved extensions. The implementation's documented extension model is individual-student based. The later whole-class extension request in the assignment brief is a maintenance change request and is not claimed as implemented without corresponding source and regression evidence.

3.1 Core functional requirements

Requirement group	System behavior / verification focus
FR1–FR2: Identity and roles	Authenticate registered users and enforce STUDENT, LECTURER, HOD and ADMIN permissions.
FR3–FR6: Administration	Manage accounts, departments and courses; assign lecturers; enroll students.
FR7–FR11: Assignment work	Show enrolled courses and assignments; submit/resubmit; timestamp submissions; apply deadlines, late penalties and individual extensions.
FR12–FR16: Assessment control	Record marks and feedback; moderate and release results; restrict unreleased marks; audit every mark change.

3.2 Non-functional requirements

The SRS makes quality attributes measurable. Key targets include 95% of assignment-list and released-result retrievals within two seconds with 800 student records; salted password hashes with no plaintext credentials; 100% denial of unauthorized access attempts in the role test suite; integrity across 100 consecutive persistence operations; offline completion of core workflows; and lecturer usability measured through representative task completion. These are acceptance targets, not measured achievements unless supported by test evidence.

3.3 Use-case model and business rules

The three principal fully dressed use cases are Submit Assignment (Student), Mark Submission (Lecturer), and Moderate and Release Marks (HoD). Their main, alternative and exception flows define the basis for integration and acceptance scenarios. Core rules require enrolment before submission, resubmission before the applicable deadline, a 10% of maximum-mark deduction per late day for up to three days, zero after three days, and application of an approved individual extension. Marks must be within the assignment range, accompanied by feedback, audited on change and hidden until release.

Figure 3. Use-case diagram for the four system actors. View diagram

Traceability is maintained from each requirement to its use case, service or UI responsibility, database entities and relevant automated test. The final implementation evidence is strongest for authentication, course services, database queries, late-penalty calculation, marking and the assessment/audit integration workflows.

4. Software architecture and detailed design

The design specifies a four-layer architecture: Presentation, Application, Domain and Data Access, backed by SQLite persistence. Qt Widgets provide the desktop presentation. Application services coordinate authentication, administration, course/enrolment, assignment, submission, marking, moderation and release use cases. Domain logic contains role permissions, assessment policies and submission-state rules. The data-access layer performs parameterized SQLite operations and transactions.

A relaxed layering rule permits the Application layer to access Domain and Data Access directly, while the Domain layer does not depend on Data Access. This keeps business rules independent of database implementation and makes services easier to test. A client-server architecture was rejected because it adds network and server administration overhead not justified by the local/offline scope. MVC can structure the UI but does not, by itself, define the business and persistence boundaries.

Figure 4. Four-layer architecture and dependency direction. View diagram

Figure 5. Component diagram and principal service responsibilities. View diagram

4.1 Object-Oriented design and design patterns

The class design uses an abstract User concept with Student, Lecturer, HoD and Administrator specializations. The documented patterns are: Strategy, through ILatePenaltyStrategy and StandardLatePenalty, to isolate the late-penalty algorithm; Repository/DAO, through interfaces such as IUserRepository, ISubmissionRepository and IMarkRepository, to isolate persistence; and Factory, through IUserFactory/UserFactory, to centralize role-specific object creation. The design applies SOLID as a set of constraints: services have focused responsibilities (SRP), alternative penalty rules can be added without rewriting callers (OCP), role implementations honor the User contract (LSP), repository interfaces remain responsibility-specific (ISP), and application logic depends on abstractions where implemented (DIP).

Figure 6. Domain class diagram, inheritance and principal interfaces. View diagram

The design also documents the submission lifecycle submitted, late, marked, moderated and released and distinguishes submission status from moderation decisions. Resubmission creates a new version and preserves the earlier record. The activity and sequence models identify the points where authentication, enrolment, deadline validation, mark validation, moderation authority and release checks are applied.

Figure 7. Submission state machine. View diagram

Figure 8. Activity diagram: submission through mark release. View diagram

Figure 9. Sequence diagram: Submit Assignment. View diagram

Figure 10. Sequence diagram: Moderate and Release Marks. View diagram

5. Implementation, database and security

The implementation is a C++17 desktop application using Qt 6 Widgets and SQLite. CMake/Ninja manages compilation, while GCC builds the application and test executables. The database persists users, departments, courses, course-lecturer relationships, enrolments, assignments, individual extensions, submission versions, marks, moderation records and mark-audit entries. Foreign keys and constraints support referential integrity; parameterized SQL reduces injection risk. The implementation separates UI windows, services and database operations, including role-specific Student, Lecturer, HoD and Administrator workflows.

Authentication uses Argon2id password hashing rather than storing plaintext passwords. Role checks protect administrative, marking and moderation operations. The mark audit records the acting user, timestamp, previous and new mark, and reason. SQLite transactions are used where multi-step persistence must remain consistent. Operational security still requires restricted access to the database file, secure backups, appropriate handling of submitted-file references and removal of demonstration credentials before real deployment.

Figure 11. Entity Relationship Diagram (ERD) and principal data relationships. View diagram

5.1 Business rules and data integrity

- A student can submit only to an assignment associated with an enrolled course; valid resubmissions are stored as successive versions.
- The applicable deadline includes an approved individual extension. Late penalties are calculated against that deadline: 10% of the maximum mark per late day for up to three days, then zero.
- Lecturers mark authorized submissions and supply written feedback. Superseded submissions are not treated as the current version for marking.
- HoDs review marks for their department. Returned marks remain unreleased; approval and release are separate actions.
- Students can retrieve released results and feedback, not marks awaiting moderation or release.

5.2 Requirements-to-design traceability

Capability	Design / implementation elements	Evidence
Authentication and roles	AuthService, PasswordUtils, user roles	7 AuthService tests; 100% file coverage
Course administration	CourseService, Database, Qt administration window	24 CourseService tests; admin GUI manual tests
Submission and deadlines	Database submission workflow, LatePenaltyStrategy	26 database-query and 9 penalty tests; workflow integration
Marking and feedback	MarkingService, marks and mark_audit tables	Marking/database tests; workflow integration
Moderation and release	HOD moderation and release database operations	Workflow integration test; audit integration test

Repository: GitHub – LMS_Project

The implementation evidence supplied for this report identifies the repository but does not independently establish a tagged v1.0/v1.1 deployment package, a completed CI workflow, static-analysis results or measured complexity/LOC metrics. These should be added only when the corresponding repository artefacts and successful run evidence are available.

6. Verification, validation and quality assurance

Testing follows the V-Model: unit tests verify individual services and business rules; integration tests verify service/database collaboration; system tests verify the integrated desktop application and role workflows; and acceptance tests evaluate the Phase 2 use cases against stakeholder needs. Entry criteria require approved specifications and implemented components; exit criteria require passing tests or documented defects, consistent database state and no unresolved critical failures.

Test level	Artefact verified	Entry / exit criteria
Unit	AuthService, CourseService, late-penalty and database query logic	Code and test cases available; all required tests pass or failures are logged.
Integration	Database/service workflows, submission, marking, moderation and audit	Unit-tested components integrated; workflow and audit state are consistent.
System	Qt application, login, role dashboards and management workflows	Deployable build available; key role and exception scenarios pass.
Acceptance	FRs and Submit, Mark, Moderate/Release use cases	Scenarios prepared; user criteria evaluated and limitations recorded.

6.1 Automated results and coverage

Suite	Passed	Failed	Total
Authentication	7	0	7
CourseService	24	0	24
Late-penalty strategy	9	0	9
Database queries	26	0	26
Integration workflows	2	0	2
TOTAL	68	0	68

The recorded CTest execution completed with 68/68 automated tests passing. GCC coverage and gcovr measured 1282 executed lines out of 1639 business-logic lines: 78% line coverage, exceeding the 70% target. Per-file results were AuthService 100%, CourseService 100%, Database 76%, LatePenaltyStrategy 96%, MarkingService 74% and PasswordUtils 79%. Coverage measures execution, not correctness or complete requirements coverage.

Integration coverage includes user/course setup, enrolment, assignment creation, extensions, submission and resubmission, attempt limits, superseded submissions, marking, HoD return/correction/approval/release, student result retrieval, late penalties and audit access controls. Manual GUI testing confirmed administrator login and access to the administration interface (SYS-01 and SYS-02: PASS). Other role-specific GUI scenarios are not reported as manually passed solely because their backend workflows have integration coverage.

6.2 System/acceptance gaps and defect log

Exception evidence includes rejection of unenrolled submissions, excess submission attempts, marking of superseded submissions and unauthorized audit access. Remaining GUI checks include student unreleased-mark visibility, HoD department-scope enforcement, invalid-mark handling and database-failure presentation. The supplied V&V artefact records no confirmed product defect in the final 68-test run; an earlier test-fixture issue was corrected and the tests passed on rerun. No unsupported defect is invented for this report.

Acceptance scenarios are defined for all four roles, but the supplied evidence does not include a complete, independently recorded user-acceptance session or stakeholder sign-off. CI execution, static analysis, 800-student performance, offline operation on target school hardware, and independent-machine deployment also remain to be evidenced. These are explicit limitations, not claimed passes.

7. Configuration management, deployment and maintenance

The project uses Git and a hosted repository for source control. The final deliverable should retain meaningful commits, a documented branch/merge strategy and tagged releases. CMake/CTest provide repeatable local build and test commands; however, local test execution is not proof of continuous integration. A successful GitHub Actions workflow, static-analysis report (for example cppcheck or clang-tidy), code metrics and self-review checklist must be attached before those Phase 6 criteria can be marked complete.

The intended deployment is a Windows desktop application with Qt runtime dependencies and a local SQLite database. Installation must verify runtime libraries, database location, file permissions, backup/restore and the availability of submitted work referenced by file paths. The single-file/local deployment approach is not evidence of safe concurrent multi-computer use. Maintenance request CR1 class-wide deadline extensions displayed in student assignment lists is a perfective change in the assignment brief. Its impact includes lecturer UI, assignment/deadline services, enrolment selection, extension persistence, student assignment retrieval and regression tests. The supplied artefacts do not establish a completed CR1 release, so it is treated as planned maintenance rather than completed functionality.

8. Professional, legal and ethical considerations

The LMS processes learner identities, credentials and assessment outcomes. Access must follow least privilege; passwords must remain salted hashes; database backups and file references must be protected; and mark changes must be explainable through audit history. Applicable Lesotho data-protection and education obligations should be confirmed by the institution before production use. The UI should support readable text, keyboard navigation and clear error feedback. Late penalties and extension decisions should be transparent, consistently applied and reviewable to reduce unfair outcomes. Third-party Qt and Argon2 dependencies must be distributed in accordance with their licenses.

9. Conclusion

The project has produced a functioning C++/Qt/SQLite LMS with four role areas, persistent assessment data, moderated mark release and an auditable marking workflow. The strongest verification evidence is the 68 passing automated tests and 78% business-logic line coverage, supported by two backend integration workflows and two manually passed administrator GUI checks. The principal remaining work is to complete broader GUI acceptance evidence, confirm CI and quality-assurance artefacts, verify performance/offline deployment constraints, and evidence CR1 before claiming full lifecycle completion.

10. References

- Talreja, A. (2025, August 5). V-Model in software development: Complete guide to verification and validation SDLC. Teaching Agile. https://www.teachingagile.com/about
- Sommerville, I. (2015) Software engineering. 10th edn. Boston: Pearson.
- GeeksforGeeks. (2026, July 7). *Waterfall Model - Software Engineering*. https://www.geeksforgeeks.org/software-engineering/waterfall-model